# Python Functions: Arguments, Returns and Default Value Pitfalls

## Defining and calling functions

A function is defined with the def keyword, a name, parentheses with parameters, and a colon. The body is indented. A function without a return statement returns None.

```
def add(a, b):
    return a + b

def greet(name):
    print("Hello", name)    # no return, so result is None

x = greet("Ravi")           # x is None
```

A very common bug is printing a value inside a function instead of returning it. The caller then receives None and fails later with a TypeError such as unsupported operand type(s) for +: 'NoneType' and 'int'.

## The mutable default argument trap

Default parameter values are evaluated once, when the function is defined, not each time it is called. If the default is a mutable object such as a list or dict, the same object is shared by every call that omits the argument, so data from earlier calls leaks into later ones.

```
def add_item(item, items=[]):     # BUG: one shared list
    items.append(item)
    return items

print(add_item("a"))   # ['a']
print(add_item("b"))   # ['a', 'b']  unexpected!

def add_item(item, items=None):   # FIX
    if items is None:
        items = []
    items.append(item)
    return items
```

The standard fix is to use None as the default and create a new list inside the function body. Immutable defaults such as numbers, strings, tuples and None are safe.

## Positional, keyword and variable arguments

Positional arguments are matched by order. Keyword arguments are matched by name. *args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dict. Calling a function with too few or too many arguments raises a TypeError that names the function and the missing parameter.

```
def info(name, age=18, *args, **kwargs):
    print(name, age, args, kwargs)

info("Asha")
info("Asha", 20, "x", city="Pune")
```

## Variable scope

Variables created inside a function are local and disappear when it returns. Reading a global variable inside a function works, but assigning to it creates a new local variable unless you declare it with the global keyword. Assigning to a name later in the function makes Python treat that name as local for the whole function, which causes UnboundLocalError when it is read before assignment.

```
counter = 0
def increment():
    counter += 1        # UnboundLocalError

def increment():
    global counter      # works, but returning a value is cleaner
    counter += 1
```

## Recursion

A recursive function calls itself. It must have a base case that stops the recursion and each call must move closer to that base case, otherwise Python raises RecursionError after about 1000 nested calls.

```
def factorial(n):
    if n <= 1:          # base case
        return 1
    return n * factorial(n - 1)
```